

Universidad de Monterrey



Integración de aplicaciones computacionales

Tarea 6

Aplicación en consola de Redis vs MariaDB

Maestro: Dr. Raúl Morales Salcedo

Nombre: Margarita Concepción Cuervo Citalán #581771

Carrera: ITC, 9no Semestre

Día y hora del grupo de la clase: lunes y jueves 10:00h

12 de octubre de 2025.

Doy mi palabra de haber realizado esta actividad con integridad académica.

ÍNDICE

Introducción	3
Desarrollo	3
Conclusión	32

Introducción

El propósito de esta tarea fue desarrollar una aplicación en consola que permita comparar el rendimiento entre dos sistemas de gestión de bases de datos: Redis y MariaDB. A través de una simulación de registro y autenticación de usuarios, se buscó medir y analizar los tiempos de respuesta de cada sistema durante operaciones de inserción y lectura. Para lograrlo, se implementó un programa en Python que integra ambas tecnologías y registra métricas detalladas. Esta actividad no solo permitió fortalecer habilidades técnicas relacionadas con bases de datos y programación en Python, sino también comprender las ventajas y limitaciones de cada sistema desde una perspectiva práctica, especialmente en escenarios donde la velocidad y eficiencia del acceso a datos son factores críticos.

Desarrollo

Se empezó esta tarea por abrir nuestras máquinas virtuales en GCP y crear el directorio *ComparacionRedis* con el comando **mkdir ComparacionRedis**. Dentro de este directorio se encuentra el archivo *main.py* el cual contendrá el código que nos entregara la IA.

```
[m03681557@maquina01 ~]$ tree .
.
├── ComparacionRedis
│   ├── main.py
│   ├── __pycache__
│   └── main.cpython-39.pyc
```

Luego se le pidió a la IA lo siguiente:

Crea un programa en Python que:

- 1. solicite al usuario el username y contraseña utilizando la tabla Users.*
- 2. pueda registrar usuarios y almacenarlos en Redis y MariaDB*
- 3. guarde y recupere todos los datos usando Redis con las librerías redis-py, MariaDB con flask_mariadb.*
- 4. Mide el tiempo de inserción y lectura en ambos sistemas usando time.time()*
- 5. muestre una comparación detallada de los tiempos, indicando cuántas veces más lento es MariaDB respecto a Redis*
- 6. Incluye el uso de Hashes para almacenar los datos del usuario*
- 7. Muestra en todo momento en el log o consola las operaciones realizadas en cada sistema junto con sus tiempos*
- 8. Explica brevemente porque Redis suele ser más rápido que otro DBMS*
- 9. Asegúrate que el programa se ejecute correctamente e incorpora comentarios útiles.*
- 10. Los datos de acceso a MariaDB son: DB:Libros, usuario: libros_user, password: 666.*

MariaDB [Libros]> DESC Users;

+-----+-----+-----+-----+-----+-----+					
Field	Type	Null	Key	Default	Extra
+-----+-----+-----+-----+-----+-----+					

<i>id</i>	<i>int(11)</i>	<i>NO</i>	<i>PRI</i>	<i>NULL</i>	<i>auto_increment</i>
<i>username</i>	<i>varchar(50)</i>	<i>NO</i>	<i>UNI</i>	<i>NULL</i>	
<i>password_hash</i>	<i>varchar(255)</i>	<i>NO</i>		<i>NULL</i>	

+-----+-----+-----+-----+-----+-----+

Tras ejecutar este prompt, la IA me dio el código del archivo **main.py**:

```
Python
"""
Sistema de Gestión de Usuarios con Comparación de Rendimiento
Redis vs MariaDB

¿Por qué Redis es más rápido?
-----

Redis es generalmente más rápido que MariaDB por estas razones:
1. Almacenamiento en memoria (RAM) vs disco (MariaDB usa disco principalmente)
2. Estructura de datos simple y optimizada (key-value store)
3. No tiene overhead de SQL parsing ni query optimization
4. No mantiene transacciones ACID complejas por defecto
5. Arquitectura single-threaded que evita locks complejos
6. No hay índices que mantener ni joins que procesar
"""

import redis
import hashlib
import time
from flask import Flask
from flask_mysql import MySQL
import sys

# Configuración de Flask y MariaDB
app = Flask(__name__)
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'libros_user'
app.config['MYSQL_PASSWORD'] = '666'
app.config['MYSQL_DB'] = 'Libros'

mysql = MySQL(app)

# Configuración de Redis
try:
    redis_client = redis.Redis(host='localhost', port=6379, db=0,
decode_responses=True)
    redis_client.ping()
    print("✓ Conexión exitosa a Redis")
except redis.ConnectionError:
    print("✗ Error: No se pudo conectar a Redis. Asegúrate de que Redis esté corriendo.")
    sys.exit(1)
```

```

def hash_password(password):
    """Genera un hash SHA-256 de la contraseña"""
    return hashlib.sha256(password.encode()).hexdigest()

def registrar_usuario_redis(username, password):
    """
    Registra un usuario en Redis usando Hashes
    Clave: user:{username}
    Hash: {username, password_hash, created_at}
    """
    print(f"\n{'='*60}")
    print(f"REDIS - Iniciando registro de usuario: {username}")

    # Verificar si el usuario ya existe
    start_check = time.time()
    existe = redis_client.exists(f"user:{username}")
    end_check = time.time()
    tiempo_check = (end_check - start_check) * 1000 # Convertir a milisegundos

    print(f"REDIS - Verificación de existencia: {tiempo_check:.4f} ms")

    if existe:
        print(f"REDIS - ✗ El usuario '{username}' ya existe")
        return False, 0

    # Insertar usuario
    password_hash = hash_password(password)
    user_data = {
        'username': username,
        'password_hash': password_hash,
        'created_at': str(int(time.time()))
    }

    start_insert = time.time()
    redis_client.hset(f"user:{username}", mapping=user_data)
    # También agregar a un conjunto para listar usuarios
    redis_client.sadd("users:all", username)
    end_insert = time.time()

    tiempo_insert = (end_insert - start_insert) * 1000
    print(f"REDIS - ✓ Usuario registrado exitosamente")
    print(f"REDIS - Tiempo de inserción: {tiempo_insert:.4f} ms")
    print(f"{'='*60}")

    return True, tiempo_insert

def registrar_usuario_mariadb(username, password):
    """Registra un usuario en MariaDB"""
    print(f"\n{'='*60}")
    print(f"MARIADB - Iniciando registro de usuario: {username}")

    with app.app_context():

```

```

cursor = mysql.connection.cursor()

# Verificar si el usuario ya existe
start_check = time.time()
cursor.execute("SELECT username FROM Users WHERE username = %s", (username,))
existe = cursor.fetchone()
end_check = time.time()
tiempo_check = (end_check - start_check) * 1000

print(f"MARIADB - Verificación de existencia: {tiempo_check:.4f} ms")

if existe:
    print(f"MARIADB - ✗ El usuario '{username}' ya existe")
    cursor.close()
    return False, 0

# Insertar usuario
password_hash = hash_password(password)

start_insert = time.time()
cursor.execute(
    "INSERT INTO Users (username, password_hash) VALUES (%s, %s)",
    (username, password_hash)
)
mysql.connection.commit()
end_insert = time.time()

tiempo_insert = (end_insert - start_insert) * 1000
print(f"MARIADB - ✓ Usuario registrado exitosamente")
print(f"MARIADB - Tiempo de inserción: {tiempo_insert:.4f} ms")
print(f"{'='*60}")

cursor.close()
return True, tiempo_insert

def autenticar_usuario_redis(username, password):
    """Autentica un usuario en Redis"""
    print(f"\n{'='*60}")
    print(f"REDIS - Iniciando autenticación de usuario: {username}")

    start_read = time.time()
    user_data = redis_client.hgetall(f"user:{username}")
    end_read = time.time()

    tiempo_read = (end_read - start_read) * 1000
    print(f"REDIS - Tiempo de lectura: {tiempo_read:.4f} ms")

    if not user_data:
        print(f"REDIS - ✗ Usuario '{username}' no encontrado")
        print(f"{'='*60}")
        return False, tiempo_read

    password_hash = hash_password(password)

```

```

if user_data.get('password_hash') == password_hash:
    print(f"REDIS - ✓ Autenticación exitosa")
    print(f"{'='*60}")
    return True, tiempo_read
else:
    print(f"REDIS - ✗ Contraseña incorrecta")
    print(f"{'='*60}")
    return False, tiempo_read

def autenticar_usuario_mariadb(username, password):
    """Autentica un usuario en MariaDB"""
    print(f"\n{'='*60}")
    print(f"MARIADB - Iniciando autenticación de usuario: {username}")

    with app.app_context():
        cursor = mysql.connection.cursor()

        start_read = time.time()
        cursor.execute(
            "SELECT password_hash FROM Users WHERE username = %s",
            (username,)
        )
        result = cursor.fetchone()
        end_read = time.time()

        tiempo_read = (end_read - start_read) * 1000
        print(f"MARIADB - Tiempo de lectura: {tiempo_read:.4f} ms")

        cursor.close()

        if not result:
            print(f"MARIADB - ✗ Usuario '{username}' no encontrado")
            print(f"{'='*60}")
            return False, tiempo_read

        password_hash = hash_password(password)
        if result[0] == password_hash:
            print(f"MARIADB - ✓ Autenticación exitosa")
            print(f"{'='*60}")
            return True, tiempo_read
        else:
            print(f"MARIADB - ✗ Contraseña incorrecta")
            print(f"{'='*60}")
            return False, tiempo_read

def mostrar_comparacion(tiempo_redis, tiempo_mariadb, operacion):
    """Muestra una comparación detallada de los tiempos"""
    print(f"\n{'#'*60}")
    print(f"COMPARACIÓN DE RENDIMIENTO - {operacion}")
    print(f"{'#'*60}")
    print(f"Redis:      {tiempo_redis:.4f} ms")
    print(f"MariaDB:    {tiempo_mariadb:.4f} ms")

```

```

print(f'{'-'*60}')

if tiempo_redis > 0:
    factor = tiempo_mariadb / tiempo_redis
    print(f"MariaDB es {factor:.2f}x más lento que Redis")
    diferencia = tiempo_mariadb - tiempo_redis
    print(f"Diferencia: {diferencia:.4f} ms")
else:
    print("Redis fue tan rápido que el tiempo fue casi 0 ms")

print(f'{'#'*60}\n")

def menu_principal():
    """Menú principal del programa"""
    with app.app_context():
        while True:
            print("\n" + "="*60)
            print("SISTEMA DE GESTIÓN DE USUARIOS - Redis vs MariaDB")
            print("="*60)
            print("1. Registrar nuevo usuario")
            print("2. Iniciar sesión (autenticar usuario)")
            print("3. Salir")
            print("="*60)

            opcion = input("Selecciona una opción (1-3): ").strip()

            if opcion == "1":
                print("\n--- REGISTRO DE USUARIO ---")
                username = input("Ingresa el nombre de usuario: ").strip()
                password = input("Ingresa la contraseña: ").strip()

                if not username or not password:
                    print("X El usuario y contraseña no pueden estar vacíos")
                    continue

                # Registrar en Redis
                exito_redis, tiempo_redis = registrar_usuario_redis(username,
password)

                # Registrar en MariaDB
                exito_mariadb, tiempo_mariadb = registrar_usuario_mariadb(username,
password)

                # Mostrar comparación si ambos fueron exitosos
                if exito_redis and exito_mariadb:
                    mostrar_comparacion(tiempo_redis, tiempo_mariadb, "INSERCIÓN")

            elif opcion == "2":
                print("\n--- AUTENTICACIÓN DE USUARIO ---")
                username = input("Ingresa el nombre de usuario: ").strip()
                password = input("Ingresa la contraseña: ").strip()

                if not username or not password:

```



```

        print("X El usuario y contraseña no pueden estar vacíos")
        continue

    # Autenticar en Redis
    exito_redis, tiempo_redis = autenticar_usuario_redis(username,
password)

    # Autenticar en MariaDB
    exito_mariadb, tiempo_mariadb = autenticar_usuario_mariadb(username,
password)

    # Mostrar comparación
    mostrar_comparacion(tiempo_redis, tiempo_mariadb, "LECTURA")

    # Verificar consistencia
    if exito_redis != exito_mariadb:
        print("⚠ ADVERTENCIA: Resultados inconsistentes entre Redis y
MariaDB")

    elif opcion == "3":
        print("\n¡Hasta luego!")
        break

    else:
        print("X Opción inválida. Por favor selecciona 1, 2 o 3.")

if __name__ == "__main__":
    print("\n" + "="*60)
    print("INICIANDO SISTEMA DE COMPARACIÓN Redis vs MariaDB")
    print("="*60)
    print("\n¿Por qué Redis es más rápido que MariaDB?")
    print("-" * 60)
    print("• Almacena datos en RAM (memoria) en lugar de disco")
    print("• Estructura de datos simple (key-value)")
    print("• No necesita parsear SQL ni optimizar queries")
    print("• No mantiene transacciones ACID complejas")
    print("• Arquitectura optimizada para operaciones rápidas")
    print("=" * 60)

    try:
        menu_principal()
    except KeyboardInterrupt:
        print("\n\n¡Programa interrumpido por el usuario!")
    except Exception as e:
        print(f"\nX Error inesperado: {e}")
        import traceback
        traceback.print_exc()

```

Tras guardar este código en el archivo **main.py**, este se ejecuta con este con el comando **python main.py**. Los resultados de dicha actividad son las siguientes:

```
ssh.cloud.google.com/v2/ssh/projects/integracion-468300/zones/us-central1-f/instances/maquina01?authuser=4&hl=es_419&projectNumber=785190282612&useAdminProxy=true - Google Chrome
ssh.cloud.google.com/v2/ssh/projects/integracion-468300/zones/us-central1-f/instances/maquina01?authuser=4&hl=es_419&projectNumber=785190282612&useAdminProxy=true

SSH en el navegador SUBIR ARCHIVO DESCARGAR ARCHIVO

[m03681557@maquina01 ComparacionRedis]$ python main.py
✓ Conexión exitosa a Redis

=====
INICIANDO SISTEMA DE COMPARACIÓN Redis vs MariaDB
=====

¿Por qué Redis es más rápido que MariaDB?
-----
• Almacena datos en RAM (memoria) en lugar de disco
• Estructura de datos simple (key-value)
• No necesita parsear SQL ni optimizar queries
• No mantiene transacciones ACID complejas
• Arquitectura optimizada para operaciones rápidas
=====

SISTEMA DE GESTIÓN DE USUARIOS - Redis vs MariaDB
-----
1. Registrar nuevo usuario
2. Iniciar sesión (autenticar usuario)
3. Salir
=====
Selecciona una opción (1-3): 1

--- REGISTRO DE USUARIO ---
Ingresa el nombre de usuario: fernanda
Ingresa la contraseña: 999

=====
REDIS - Iniciando registro de usuario: fernanda
REDIS - Verificación de existencia: 0.7100 ms
REDIS - ✓ Usuario registrado exitosamente
REDIS - Tiempo de inserción: 0.9868 ms
=====

MARIADB - Iniciando registro de usuario: fernanda
MARIADB - Verificación de existencia: 8.5087 ms
MARIADB - ✓ Usuario registrado exitosamente
```

```
ssh.cloud.google.com/v2/ssh/projects/integracion-468300/zones/us-central1-f/instances/maquina01?authuser=4&hl=es_419&projectNumber=785190282612&useAdminProxy=true - Google Chrome
ssh.cloud.google.com/v2/ssh/projects/integracion-468300/zones/us-central1-f/instances/maquina01?authuser=4&hl=es_419&projectNumber=785190282612&useAdminProxy=true

SSH en el navegador SUBIR ARCHIVO DESCARGAR ARCHIVO

REDIS - Iniciando registro de usuario: fernanda
REDIS - Verificación de existencia: 0.7100 ms
REDIS - ✓ Usuario registrado exitosamente
REDIS - Tiempo de inserción: 0.9868 ms
=====

MARIADB - Iniciando registro de usuario: fernanda
MARIADB - Verificación de existencia: 8.5087 ms
MARIADB - ✓ Usuario registrado exitosamente
MARIADB - Tiempo de inserción: 4.0979 ms
=====

#####
COMPARACIÓN DE RENDIMIENTO - INSERCIÓN
#####
Redis: 0.9868 ms
MariaDB: 4.0979 ms
=====
MariaDB es 4.15x más lento que Redis
Diferencia: 3.1111 ms
#####

SISTEMA DE GESTIÓN DE USUARIOS - Redis vs MariaDB
-----
1. Registrar nuevo usuario
2. Iniciar sesión (autenticar usuario)
3. Salir
=====
Selecciona una opción (1-3): 2

--- AUTENTICACIÓN DE USUARIO ---
Ingresa el nombre de usuario: fernanda
Ingresa la contraseña: 999

=====
REDIS - Iniciando autenticación de usuario: fernanda
```

```
ssh.cloud.google.com/v2/ssh/projects/integracion-468300/zones/us-central1-f/instances/maquina01?authuser=4&hl=es_419&projectNumber=785190282612&useAdminProxy=true - Google Chrome
ssh.cloud.google.com/v2/ssh/projects/integracion-468300/zones/us-central1-f/instances/maquina01?authuser=4&hl=es_419&projectNumber=785190282612&useAdminProxy=true

SSH en el navegador
SUBIR ARCHIVO
DESCARGAR ARCHIVO

1. Registrar nuevo usuario
2. Iniciar sesión (autenticar usuario)
3. Salir

Selecciona una opción (1-3): 2

--- AUTENTICACIÓN DE USUARIO ---
Ingresa el nombre de usuario: fernanda
Ingresa la contraseña: 999

=====
REDIS - Iniciando autenticación de usuario: fernanda
REDIS - Tiempo de lectura: 0.9854 ms
REDIS - ✓ Autenticación exitosa

=====
MARIADB - Iniciando autenticación de usuario: fernanda
MARIADB - Tiempo de lectura: 0.7274 ms
MARIADB - ✓ Autenticación exitosa

=====
#####
COMPARACIÓN DE RENDIMIENTO - LECTURA
#####
Redis: 0.9854 ms
MariaDB: 0.7274 ms

-----
MariaDB es 0.74x más lento que Redis
Diferencia: -0.2580 ms
#####

=====
SISTEMA DE GESTIÓN DE USUARIOS - Redis vs MariaDB
=====
1. Registrar nuevo usuario
2. Iniciar sesión (autenticar usuario)
3. Salir

Selecciona una opción (1-3):
```

Conclusión

Tras realizar esta actividad, me di cuenta de la importancia y la conveniencia de utilizar Redis para el registro e inicio de sesión de los usuarios. Esto se debe a que en la inserción de un nuevo usuario, Redis es 4.15 veces más rápido que MariaDB. En el caso del inicio de sesión de un usuario ya existente, Redis es 0.74 veces más rápido que MariaDB. Esto se debe a que Redis almacena todos sus datos en memoria (RAM), lo que permite operaciones de lectura y escritura casi instantáneas en comparación con las bases de datos, como MariaDB, que dependen del disco.

En conclusión, esta experiencia permitió constatar que Redis puede ser una herramienta valiosa en aplicaciones donde el rendimiento y la velocidad son esenciales, mientras que MariaDB sigue siendo una opción robusta para estructuras más complejas o donde la persistencia a largo plazo sea prioritaria.